

Лабораторная работа 2

Создано системой Doxygen 1.17.0

1 Лабораторная работа 2	1
2 Алфавитный указатель классов	3
2.1 Классы	3
3 Список файлов	5
3.1 Файлы	5
4 Классы	7
4.1 Класс Benchmark	7
4.1.1 Подробное описание	7
4.1.2 Методы	8
4.1.2.1 exportToTXT()	8
4.1.2.2 getRepeatCount()	8
4.1.2.3 runAllTests()	8
4.1.2.4 runTest()	9
4.2 Класс BinarySearchTree	9
4.2.1 Подробное описание	10
4.2.2 Конструктор(ы)	10
4.2.2.1 BinarySearchTree()	10
4.2.2.2 ~BinarySearchTree()	10
4.2.3 Методы	10
4.2.3.1 buildFromArray()	10
4.2.3.2 insert()	11
4.2.3.3 search()	11
4.2.4 Данные класса	11
4.2.4.1 root	11
4.3 Структура HashTable::Bucket	11
4.3.1 Подробное описание	12
4.3.2 Данные класса	12
4.3.2.1 capacity	12
4.3.2.2 data	12
4.3.2.3 size	12
4.4 Структура DataObject	13
4.4.1 Подробное описание	13
4.4.2 Конструктор(ы)	13
4.4.2.1 DataObject() [1/2]	13
4.4.2.2 DataObject() [2/2]	14
4.4.3 Методы	14
4.4.3.1 getKey()	14
4.4.4 Данные класса	14
4.4.4.1 field1	14
4.4.4.2 field2	14
4.4.4.3 value1	15

4.4.4.4 value2	15
4.5 Класс DynamicArray	15
4.5.1 Подробное описание	16
4.5.2 Конструктор(ы)	16
4.5.2.1 DynamicArray()	16
4.5.2.2 ~DynamicArray()	16
4.5.3 Методы	16
4.5.3.1 getSize()	16
4.5.3.2 operator[]()	17
4.5.3.3 push_back()	17
4.5.3.4 reserve()	17
4.5.4 Данные класса	17
4.5.4.1 arr	17
4.5.4.2 capacity	18
4.5.4.3 ownsObjects	18
4.5.4.4 size	18
4.6 Класс HashTable	18
4.6.1 Подробное описание	19
4.6.2 Конструктор(ы)	19
4.6.2.1 HashTable()	19
4.6.2.2 ~HashTable()	19
4.6.3 Методы	20
4.6.3.1 addToBucket()	20
4.6.3.2 buildFromArray()	20
4.6.3.3 getCollisions()	20
4.6.3.4 hashFunction()	21
4.6.3.5 initBucket()	21
4.6.3.6 insert()	21
4.6.3.7 search()	22
4.6.4 Данные класса	22
4.6.4.1 collisions	22
4.6.4.2 table	22
4.6.4.3 tableSize	22
4.7 Класс LinearSearch	23
4.7.1 Подробное описание	23
4.7.2 Методы	23
4.7.2.1 search()	23
4.8 Класс RedBlackTree	24
4.8.1 Подробное описание	24
4.8.2 Конструктор(ы)	24
4.8.2.1 RedBlackTree()	24
4.8.2.2 ~RedBlackTree()	25
4.8.3 Методы	25

4.8.3.1 buildFromArray()	25
4.8.3.2 fixInsert()	25
4.8.3.3 insert()	25
4.8.3.4 rotateLeft()	26
4.8.3.5 rotateRight()	26
4.8.3.6 search()	26
4.8.4 Данные класса	27
4.8.4.1 root	27
4.9 Структура Benchmark::Result	27
4.9.1 Подробное описание	27
4.9.2 Данные класса	28
4.9.2.1 bstTime	28
4.9.2.2 collisions	28
4.9.2.3 hashTime	28
4.9.2.4 linearTime	28
4.9.2.5 multimapTime	28
4.9.2.6 rbTreeTime	28
4.9.2.7 size	29
4.10 Структура TreeNode	29
4.10.1 Подробное описание	29
4.10.2 Конструктор(ы)	30
4.10.2.1 TreeNode()	30
4.10.2.2 ~TreeNode()	30
4.10.3 Данные класса	30
4.10.3.1 isRed	30
4.10.3.2 key	30
4.10.3.3 left	30
4.10.3.4 objects	31
4.10.3.5 parent	31
4.10.3.6 right	31
5 Файлы	33
5.1 Файл main.cpp	33
5.1.1 Подробное описание	33
5.1.2 Функции	33
5.1.2.1 main()	33
5.2 main.cpp	34
5.3 Файл page.md	34
5.4 Файл search_algorithms.cpp	34
5.4.1 Подробное описание	34
5.4.2 Функции	35
5.4.2.1 destroyTreeNodes()	35
5.4.2.2 getKeyFromData()	35

5.4.2.3 loadDataFromFile()	35
5.4.2.4 pushTreeNode()	36
5.5 search_algorithms.cpp	36
5.6 Файл search_algorithms.h	44
5.6.1 Подробное описание	45
5.6.2 Функции	45
5.6.2.1 getKeyFromData()	45
5.6.2.2 loadDataFromFile()	45
5.7 search_algorithms.h	46
Предметный указатель	49

Глава 1

Лабораторная работа 2

Цель работы:

Реализовать и сравнить методы поиска объектов по строковому ключу.

Формат входных данных:

Входные данные хранятся в файле "ds.txt", созданном при выполнении ЛР1.

Реализованные методы поиска:

1. Линейный поиск;
2. Поиск с помощью бинарного дерева поиска;
3. Поиск с помощью красно-чёрного дерева;
4. Поиск с помощью хэш-таблицы;
5. Поиск с помощью multimap

Особенности реализации:

Для хранения исходных данных используется динамический массив указателей на объекты.

Для корректной работы с неуникальными ключами в деревьях один узел хранит один ключ и массив объектов с этим ключом.

В хэш-таблице используется метод цепочек. Коллизией считается вставка объекта в уже непустую ячейку таблицы.

Результаты сохраняются в файл "res.txt"

Вывод:

Линейный поиск показывает наиболее быстрый рост времени при увеличении размера массива, так как последовательно просматривает все элементы.

Бинарное дерево, красно-чёрное дерево, хэш-таблица и multimap обеспечивают более быстрое выполнение поиска.

Глава 2

Алфавитный указатель классов

2.1 Классы

Классы с их кратким описанием.

Benchmark	
Класс тестирования алгоритмов поиска	7
BinarySearchTree	
Бинарное дерево поиска	9
HashTable::Bucket	
Ячейка хэш-таблицы	11
DataObject	
Структура объекта данных	13
DynamicArray	
Динамический массив указателей на объекты DataObject	15
HashTable	
Хэш-таблица с методом цепочек	18
LinearSearch	
Класс линейного поиска	23
RedBlackTree	
Красно-черное дерево	24
Benchmark::Result	
Структура результата тестирования	27
TreeNode	
Узел дерева поиска	29

Глава 3

Список файлов

3.1 Файлы

Полный список файлов.

main.cpp	Главный файл программы для запуска сравнения алгоритмов поиска	33
search_algorithms.cpp	Файл реализации структур данных, алгоритмов поиска и тестирования	34
search_algorithms.h	Заголовочный файл с объявлениями структур, классов и функций поиска	44

Глава 4

Классы

4.1 Класс Benchmark

Класс тестирования алгоритмов поиска.

```
#include <search_algorithms.h>
```

Классы

- `struct Result`
Структура результата тестирования.

Открытые члены

- `Result * runTest (DynamicArray *data, const string &searchKey)`
Выполняет тестирование на одном массиве.
- `void runAllTests (int sizes[], int numSizes, const string &searchKey, DynamicArray *originalData)`
Выполняет тестирование на массивах разных размеров.
- `void exportToTXT (Result **results, int count, const string &filename)`
Сохраняет результаты тестирования в текстовый файл.

Закрытые члены

- `int getRepeatCount (int size)`
Возвращает количество повторов поиска для усреднения времени.

4.1.1 Подробное описание

Класс тестирования алгоритмов поиска.

Класс выполняет построение структур данных, многократный запуск поиска, вычисление среднего времени поиска и сохранение результатов в файл `res.txt`.

См. определение в файле [search_algorithms.h](#) строка 430

4.1.2 Методы

4.1.2.1 exportToTXT()

```
void Benchmark::exportToTXT (  
    Result ** results,  
    int count,  
    const string & filename)
```

Сохраняет результаты тестирования в текстовый файл.

Аргументы

results	Массив результатов.
count	Количество результатов.
filename	Имя выходного файла.

См. определение в файле [search_algorithms.cpp](#) строка [646](#)

4.1.2.2 getRepeatCount()

```
int Benchmark::getRepeatCount (  
    int size) [private]
```

Возвращает количество повторов поиска для усреднения времени.

Аргументы

size	Размер тестового массива.
------	---------------------------

Возвращает

Количество повторов.

См. определение в файле [search_algorithms.cpp](#) строка [524](#)

4.1.2.3 runAllTests()

```
void Benchmark::runAllTests (  
    int sizes[],  
    int numSizes,  
    const string & searchKey,  
    DynamicArray * originalData)
```

Выполняет тестирование на массивах разных размеров.

Аргументы

sizes	Массив размерностей.
numSizes	Количество размерностей.
searchKey	Ключ поиска.
originalData	Исходный массив данных.

См. определение в файле [search_algorithms.cpp](#) строка 620

4.1.2.4 runTest()

```
Benchmark::Result * Benchmark::runTest (
    DynamicArray * data,
    const string & searchKey)
```

Выполняет тестирование на одном массиве.

Аргументы

data	Тестовый массив.
searchKey	Ключ поиска.

Возвращает

Указатель на структуру с результатами тестирования.

См. определение в файле [search_algorithms.cpp](#) строка 540

Объявления и описания членов классов находятся в файлах:

- [search_algorithms.h](#)
- [search_algorithms.cpp](#)

4.2 Класс BinarySearchTree

Бинарное дерево поиска.

```
#include <search_algorithms.h>
```

Открытые члены

- [BinarySearchTree](#) ()
Конструктор бинарного дерева поиска.
- [~BinarySearchTree](#) ()
Деструктор бинарного дерева поиска.
- void [insert](#) ([DataObject](#) *obj)
Добавляет объект в дерево.
- void [buildFromArray](#) ([DynamicArray](#) *arr)
Строит дерево по массиву объектов.
- [DynamicArray](#) * [search](#) (const string &key)
Выполняет поиск объектов по ключу.

Закрытые данные

- [TreeNode * root](#)
Корень дерева.

4.2.1 Подробное описание

Бинарное дерево поиска.

Класс реализует хранение объектов по строковому ключу. Повторяющиеся ключи не создают новые узлы, а сохраняются в массиве объектов внутри существующего узла.

См. определение в файле [search_algorithms.h](#) строка 218

4.2.2 Конструктор(ы)

4.2.2.1 BinarySearchTree()

```
BinarySearchTree::BinarySearchTree ()
```

Конструктор бинарного дерева поиска.

См. определение в файле [search_algorithms.cpp](#) строка 181

4.2.2.2 ~BinarySearchTree()

```
BinarySearchTree::~~BinarySearchTree ()
```

Деструктор бинарного дерева поиска.

См. определение в файле [search_algorithms.cpp](#) строка 185

4.2.3 Методы

4.2.3.1 buildFromArray()

```
void BinarySearchTree::buildFromArray (
    DynamicArray * arr)
```

Строит дерево по массиву объектов.

Аргументы

arr	Массив объектов.
-----	------------------

См. определение в файле [search_algorithms.cpp](#) строка 226

4.2.3.2 insert()

```
void BinarySearchTree::insert (  
    DataObject * obj)
```

Добавляет объект в дерево.

Аргументы

obj	Указатель на добавляемый объект.
-----	----------------------------------

См. определение в файле [search_algorithms.cpp](#) строка 189

4.2.3.3 search()

```
DynamicArray * BinarySearchTree::search (  
    const string & key)
```

Выполняет поиск объектов по ключу.

Аргументы

key	Ключ поиска.
-----	--------------

Возвращает

Массив найденных объектов.

См. определение в файле [search_algorithms.cpp](#) строка 232

4.2.4 Данные класса

4.2.4.1 root

```
TreeNode* BinarySearchTree::root [private]
```

Корень дерева.

См. определение в файле [search_algorithms.h](#) строка 221

Объявления и описания членов классов находятся в файлах:

- [search_algorithms.h](#)
- [search_algorithms.cpp](#)

4.3 Структура HashTable::Bucket

Ячейка хэш-таблицы.

Открытые атрибуты

- [DataObject](#) ** [data](#)
Массив указателей на объекты.
- `int` [size](#)
Количество объектов в ячейке.
- `int` [capacity](#)
Вместимость ячейки.

4.3.1 Подробное описание

Ячейка хэш-таблицы.

Ячейка содержит динамический массив указателей на объекты, которые попали в один и тот же индекс таблицы.

См. определение в файле [search_algorithms.h](#) строка [338](#)

4.3.2 Данные класса

4.3.2.1 capacity

```
int HashTable::Bucket::capacity
```

Вместимость ячейки.

См. определение в файле [search_algorithms.h](#) строка [346](#)

4.3.2.2 data

```
DataObject** HashTable::Bucket::data
```

Массив указателей на объекты.

См. определение в файле [search_algorithms.h](#) строка [340](#)

4.3.2.3 size

```
int HashTable::Bucket::size
```

Количество объектов в ячейке.

См. определение в файле [search_algorithms.h](#) строка [343](#)

Объявления и описания членов структуры находятся в файле:

- [search_algorithms.h](#)

4.4 Структура DataObject

Структура объекта данных.

```
#include <search_algorithms.h>
```

Открытые члены

- `DataObject ()`
Конструктор по умолчанию.
- `DataObject (const string &f1, const string &f2, int v1, int v2)`
Конструктор с параметрами.
- `string getKey () const`
Возвращает ключ поиска объекта.

Открытые атрибуты

- `string field1`
Первое строковое поле, используемое как ключ поиска.
- `string field2`
Второе строковое поле объекта.
- `int value1`
Первое числовое значение объекта.
- `int value2`
Второе числовое значение объекта.

4.4.1 Подробное описание

Структура объекта данных.

Объект содержит два строковых поля и два числовых значения. В качестве ключа поиска используется первое строковое поле

См. определение в файле [search_algorithms.h](#) строка 51

4.4.2 Конструктор(ы)

4.4.2.1 DataObject() [1/2]

```
DataObject::DataObject ()
```

Конструктор по умолчанию.

См. определение в файле [search_algorithms.cpp](#) строка 15

4.4.2.2 DataObject() [2/2]

```
DataObject::DataObject (  
    const string & f1,  
    const string & f2,  
    int v1,  
    int v2)
```

Конструктор с параметрами.

Аргументы

f1	Первое строковое поле.
f2	Второе строковое поле.
v1	Первое числовое значение.
v2	Второе числовое значение.

См. определение в файле [search_algorithms.cpp](#) строка 18

4.4.3 Методы

4.4.3.1 getKey()

```
string DataObject::getKey () const
```

Возвращает ключ поиска объекта.

Возвращает

Значение поля field1.

См. определение в файле [search_algorithms.cpp](#) строка 22

4.4.4 Данные класса

4.4.4.1 field1

```
string DataObject::field1
```

Первое строковое поле, используемое как ключ поиска.

См. определение в файле [search_algorithms.h](#) строка 53

4.4.4.2 field2

```
string DataObject::field2
```

Второе строковое поле объекта.

См. определение в файле [search_algorithms.h](#) строка 56

4.4.4.3 value1

```
int DataObject::value1
```

Первое числовое значение объекта.

См. определение в файле [search_algorithms.h](#) строка 59

4.4.4.4 value2

```
int DataObject::value2
```

Второе числовое значение объекта.

См. определение в файле [search_algorithms.h](#) строка 62

Объявления и описания членов структур находятся в файлах:

- [search_algorithms.h](#)
- [search_algorithms.cpp](#)

4.5 Класс DynamicArray

Динамический массив указателей на объекты [DataObject](#).

```
#include <search_algorithms.h>
```

Открытые члены

- [DynamicArray](#) (int initialCapacity=10, bool ownsObjects=false)
Конструктор динамического массива.
- [~DynamicArray](#) ()
Деструктор динамического массива.
- void [push_back](#) ([DataObject](#) *obj)
Добавляет объект в конец массива.
- void [reserve](#) (int newCapacity)
Резервирует новую вместимость массива.
- [DataObject](#) * [operator\[\]](#) (int index)
Возвращает элемент массива по индексу.
- int [getSize](#) () const
Возвращает количество элементов массива.

Закрытые данные

- [DataObject](#) ** [arr](#)
Массив указателей на объекты.
- int [capacity](#)
Вместимость массива.
- int [size](#)
Текущее количество элементов.
- bool [ownsObjects](#)
Признак владения объектами внутри массива.

4.5.1 Подробное описание

Динамический массив указателей на объекты [DataObject](#).

Класс используется для хранения исходных данных, промежуточных массивов, результатов поиска и списков объектов с одинаковыми ключами внутри узлов деревьев. Поддерживается ручное управление памятью.

См. определение в файле [search_algorithms.h](#) строка 94

4.5.2 Конструктор(ы)

4.5.2.1 DynamicArray()

```
DynamicArray::DynamicArray (
    int initialCapacity = 10,
    bool ownsObjects = false)
```

Конструктор динамического массива.

Аргументы

initialCapacity	Начальная вместимость массива.
ownsObjects	Признак владения объектами.

См. определение в файле [search_algorithms.cpp](#) строка 26

4.5.2.2 ~DynamicArray()

```
DynamicArray::~DynamicArray ()
```

Деструктор динамического массива.

Если массив владеет объектами, то деструктор освобождает память, выделенную под объекты [DataObject](#).

См. определение в файле [search_algorithms.cpp](#) строка 33

4.5.3 Методы

4.5.3.1 getSize()

```
int DynamicArray::getSize () const
```

Возвращает количество элементов массива.

Возвращает

Количество элементов.

См. определение в файле [search_algorithms.cpp](#) строка 84

4.5.3.2 operator[]()

```
DataObject * DynamicArray::operator[] (
    int index)
```

Возвращает элемент массива по индексу.

Аргументы

index	Индекс элемента.
-------	------------------

Возвращает

Указатель на объект [DataObject](#).

См. определение в файле [search_algorithms.cpp](#) строка 80

4.5.3.3 push_back()

```
void DynamicArray::push_back (
    DataObject * obj)
```

Добавляет объект в конец массива.

Аргументы

obj	Указатель на добавляемый объект.
-----	----------------------------------

См. определение в файле [search_algorithms.cpp](#) строка 60

4.5.3.4 reserve()

```
void DynamicArray::reserve (
    int newCapacity)
```

Резервирует новую вместимость массива.

Аргументы

newCapacity	Новая вместимость массива.
-------------	----------------------------

См. определение в файле [search_algorithms.cpp](#) строка 43

4.5.4 Данные класса

4.5.4.1 arr

```
DataObject** DynamicArray::arr [private]
```

Массив указателей на объекты.

См. определение в файле [search_algorithms.h](#) строка 97

4.5.4.2 capacity

`int DynamicArray::capacity [private]`

Вместимость массива.

См. определение в файле [search_algorithms.h](#) строка 100

4.5.4.3 ownsObjects

`bool DynamicArray::ownsObjects [private]`

Признак владения объектами внутри массива.

См. определение в файле [search_algorithms.h](#) строка 106

4.5.4.4 size

`int DynamicArray::size [private]`

Текущее количество элементов.

См. определение в файле [search_algorithms.h](#) строка 103

Объявления и описания членов классов находятся в файлах:

- [search_algorithms.h](#)
- [search_algorithms.cpp](#)

4.6 Класс HashTable

Хэш-таблица с методом цепочек.

```
#include <search_algorithms.h>
```

Классы

- `struct Bucket`
Ячейка хэш-таблицы.

Открытые члены

- `HashTable (int size)`
Конструктор хэш-таблицы.
- `~HashTable ()`
Деструктор хэш-таблицы.
- `void insert (DataObject *obj)`
Добавляет объект в хэш-таблицу.
- `void buildFromArray (DynamicArray *arr)`
Строит хэш-таблицу по массиву объектов.
- `DynamicArray * search (const string &key)`
Выполняет поиск объектов по ключу.
- `int getCollisions () const`
Возвращает количество коллизий.

Закрытые члены

- `int hashFunction (const string &key) const`
Вычисляет индекс по строковому ключу.
- `void initBucket (Bucket &bucket)`
Инициализирует ячейку хэш-таблицы.
- `void addToBucket (Bucket &bucket, DataObject *obj)`
Добавляет объект в ячейку хэш-таблицы.

Закрытые данные

- `Bucket * table`
Массив ячеек хэш-таблицы.
- `int tableSize`
Размер хэш-таблицы.
- `int collisions`
Количество коллизий.

4.6.1 Подробное описание

Хэш-таблица с методом цепочек.

Класс реализует собственную хэш-функцию и хранение объектов в ячейках таблицы. Коллизии разрешаются методом цепочек с использованием динамических массивов внутри ячеек.

См. определение в файле [search_algorithms.h](#) строка 330

4.6.2 Конструктор(ы)

4.6.2.1 HashTable()

```
HashTable::HashTable (
    int size)
```

Конструктор хэш-таблицы.

Аргументы

size	Размер хэш-таблицы.
------	---------------------

См. определение в файле [search_algorithms.cpp](#) строка 431

4.6.2.2 ~HashTable()

```
HashTable::~HashTable ()
```

Деструктор хэш-таблицы.

См. определение в файле [search_algorithms.cpp](#) строка 441

4.6.3 Методы

4.6.3.1 addToBucket()

```
void HashTable::addToBucket (
    Bucket & bucket,
    DataObject * obj) [private]
```

Добавляет объект в ячейку хэш-таблицы.

Аргументы

bucket	Ячейка таблицы.
obj	Указатель на добавляемый объект.

См. определение в файле [search_algorithms.cpp](#) строка 455

4.6.3.2 buildFromArray()

```
void HashTable::buildFromArray (
    DynamicArray * arr)
```

Строит хэш-таблицу по массиву объектов.

Аргументы

arr	Массив объектов.
-----	------------------

См. определение в файле [search_algorithms.cpp](#) строка 500

4.6.3.3 getCollisions()

```
int HashTable::getCollisions () const
```

Возвращает количество коллизий.

Возвращает

Количество коллизий хэш-функции.

См. определение в файле [search_algorithms.cpp](#) строка 520

4.6.3.4 hashFunction()

```
int HashTable::hashFunction (  
    const string & key) const    [private]
```

Вычисляет индекс по строковому ключу.

Аргументы

key	Ключ объекта.
-----	---------------

Возвращает

Индекс ячейки хэш-таблицы.

См. определение в файле [search_algorithms.cpp](#) строка 480

4.6.3.5 initBucket()

```
void HashTable::initBucket (  
    Bucket & bucket)    [private]
```

Инициализирует ячейку хэш-таблицы.

Аргументы

bucket	Ячейка таблицы.
--------	-----------------

См. определение в файле [search_algorithms.cpp](#) строка 449

4.6.3.6 insert()

```
void HashTable::insert (  
    DataObject * obj)
```

Добавляет объект в хэш-таблицу.

Аргументы

obj	Указатель на добавляемый объект.
-----	----------------------------------

См. определение в файле [search_algorithms.cpp](#) строка 490

4.6.3.7 search()

```
DynamicArray * HashTable::search (  
    const string & key)
```

Выполняет поиск объектов по ключу.

Аргументы

key	Ключ поиска.
-----	--------------

Возвращает

Массив найденных объектов.

См. определение в файле [search_algorithms.cpp](#) строка [506](#)

4.6.4 Данные класса

4.6.4.1 collisions

```
int HashTable::collisions [private]
```

Количество коллизий.

См. определение в файле [search_algorithms.h](#) строка [356](#)

4.6.4.2 table

```
Bucket* HashTable::table [private]
```

Массив ячеек хэш-таблицы.

См. определение в файле [search_algorithms.h](#) строка [350](#)

4.6.4.3 tableSize

```
int HashTable::tableSize [private]
```

Размер хэш-таблицы.

См. определение в файле [search_algorithms.h](#) строка [353](#)

Объявления и описания членов классов находятся в файлах:

- [search_algorithms.h](#)
- [search_algorithms.cpp](#)

4.7 Класс LinearSearch

Класс линейного поиска.

```
#include <search_algorithms.h>
```

Открытые статические члены

- static [DynamicArray](#) * [search](#) ([DynamicArray](#) *arr, const string &key)
Выполняет линейный поиск по массиву.

4.7.1 Подробное описание

Класс линейного поиска.

Выполняет последовательный просмотр всех элементов массива и возвращает все объекты, ключ которых совпадает с заданным.

См. определение в файле [search_algorithms.h](#) строка 199

4.7.2 Методы

4.7.2.1 search()

```
DynamicArray * LinearSearch::search (  
    DynamicArray * arr,  
    const string & key) [static]
```

Выполняет линейный поиск по массиву.

Аргументы

arr	Массив объектов.
key	Ключ поиска.

Возвращает

Массив найденных объектов.

См. определение в файле [search_algorithms.cpp](#) строка 169

Объявления и описания членов классов находятся в файлах:

- [search_algorithms.h](#)
- [search_algorithms.cpp](#)

4.8 Класс RedBlackTree

Красно-черное дерево.

```
#include <search_algorithms.h>
```

Открытые члены

- [RedBlackTree](#) ()
Конструктор красно-черного дерева.
- [~RedBlackTree](#) ()
Деструктор красно-черного дерева.
- void [insert](#) ([DataObject](#) *obj)
Добавляет объект в красно-черное дерево.
- void [buildFromArray](#) ([DynamicArray](#) *arr)
Строит красно-черное дерево по массиву объектов.
- [DynamicArray](#) * [search](#) (const string &key)
Выполняет поиск объектов по ключу.

Закрытые члены

- void [rotateLeft](#) ([TreeNode](#) *x)
Выполняет левый поворот.
- void [rotateRight](#) ([TreeNode](#) *x)
Выполняет правый поворот.
- void [fixInsert](#) ([TreeNode](#) *z)
Восстанавливает свойства красно-черного дерева после вставки.

Закрытые данные

- [TreeNode](#) * [root](#)
Корень дерева.

4.8.1 Подробное описание

Красно-черное дерево.

Класс реализует самобалансирующееся бинарное дерево поиска. Повторяющиеся ключи сохраняются в одном узле дерева.

См. определение в файле [search_algorithms.h](#) строка 263

4.8.2 Конструктор(ы)

4.8.2.1 RedBlackTree()

```
RedBlackTree::RedBlackTree ()
```

Конструктор красно-черного дерева.

См. определение в файле [search_algorithms.cpp](#) строка 257

4.8.2.2 ~RedBlackTree()

```
RedBlackTree::~RedBlackTree ()
```

Деструктор красно-черного дерева.

См. определение в файле [search_algorithms.cpp](#) строка 261

4.8.3 Методы

4.8.3.1 buildFromArray()

```
void RedBlackTree::buildFromArray (  
    DynamicArray * arr)
```

Строит красно-черное дерево по массиву объектов.

Аргументы

arr	Массив объектов.
-----	------------------

См. определение в файле [search_algorithms.cpp](#) строка 400

4.8.3.2 fixInsert()

```
void RedBlackTree::fixInsert (  
    TreeNode * z) [private]
```

Восстанавливает свойства красно-черного дерева после вставки.

Аргументы

z	Добавленный узел.
---	-------------------

См. определение в файле [search_algorithms.cpp](#) строка 315

4.8.3.3 insert()

```
void RedBlackTree::insert (  
    DataObject * obj)
```

Добавляет объект в красно-черное дерево.

Аргументы

obj	Указатель на добавляемый объект.
-----	----------------------------------

См. определение в файле [search_algorithms.cpp](#) строка 362

4.8.3.4 rotateLeft()

```
void RedBlackTree::rotateLeft (  
    TreeNode * x) [private]
```

Выполняет левый поворот.

Аргументы

x	Узел, относительно которого выполняется поворот.
---	--

См. определение в файле [search_algorithms.cpp](#) строка 265

4.8.3.5 rotateRight()

```
void RedBlackTree::rotateRight (  
    TreeNode * x) [private]
```

Выполняет правый поворот.

Аргументы

x	Узел, относительно которого выполняется поворот.
---	--

См. определение в файле [search_algorithms.cpp](#) строка 290

4.8.3.6 search()

```
DynamicArray * RedBlackTree::search (  
    const string & key)
```

Выполняет поиск объектов по ключу.

Аргументы

key	Ключ поиска.
-----	--------------

Возвращает

Массив найденных объектов.

См. определение в файле [search_algorithms.cpp](#) строка 406

4.8.4 Данные класса

4.8.4.1 root

`TreeNode* RedBlackTree::root` [private]

Корень дерева.

См. определение в файле [search_algorithms.h](#) строка 266

Объявления и описания членов классов находятся в файлах:

- [search_algorithms.h](#)
- [search_algorithms.cpp](#)

4.9 Структура Benchmark::Result

Структура результата тестирования.

```
#include <search_algorithms.h>
```

Открытые атрибуты

- `int size`
Размер тестового массива.
- `double linearTime`
Среднее время линейного поиска.
- `double bstTime`
Среднее время поиска в бинарном дереве.
- `double rbTreeTime`
Среднее время поиска в красно-черном дереве.
- `double hashTime`
Среднее время поиска в хэш-таблице.
- `double multimapTime`
Среднее время поиска в multimap.
- `int collisions`
Количество коллизий хэш-функции.

4.9.1 Подробное описание

Структура результата тестирования.

Хранит размер массива, время поиска каждым методом и количество коллизий.

См. определение в файле [search_algorithms.h](#) строка 446

4.9.2 Данные класса

4.9.2.1 bstTime

`double Benchmark::Result::bstTime`

Среднее время поиска в бинарном дереве.

См. определение в файле [search_algorithms.h](#) строка 454

4.9.2.2 collisions

`int Benchmark::Result::collisions`

Количество коллизий хэш-функции.

См. определение в файле [search_algorithms.h](#) строка 466

4.9.2.3 hashTime

`double Benchmark::Result::hashTime`

Среднее время поиска в хэш-таблице.

См. определение в файле [search_algorithms.h](#) строка 460

4.9.2.4 linearTime

`double Benchmark::Result::linearTime`

Среднее время линейного поиска.

См. определение в файле [search_algorithms.h](#) строка 451

4.9.2.5 multimapTime

`double Benchmark::Result::multimapTime`

Среднее время поиска в multimap.

См. определение в файле [search_algorithms.h](#) строка 463

4.9.2.6 rbTreeTime

`double Benchmark::Result::rbTreeTime`

Среднее время поиска в красно-черном дереве.

См. определение в файле [search_algorithms.h](#) строка 457

4.9.2.7 size

```
int Benchmark::Result::size
```

Размер тестового массива.

См. определение в файле [search_algorithms.h](#) строка 448

Объявления и описания членов структуры находятся в файле:

- [search_algorithms.h](#)

4.10 Структура TreeNode

Узел дерева поиска.

```
#include <search_algorithms.h>
```

Открытые члены

- [TreeNode](#) ([DataObject](#) *obj)
Конструктор узла дерева.
- [~TreeNode](#) ()
Деструктор узла дерева.

Открытые атрибуты

- [string](#) [key](#)
Ключ узла.
- [DynamicArray](#) * [objects](#)
Массив объектов, имеющих одинаковый ключ.
- [TreeNode](#) * [left](#)
Левый потомок.
- [TreeNode](#) * [right](#)
Правый потомок.
- [TreeNode](#) * [parent](#)
Родительский узел.
- [bool](#) [isRed](#)
Цвет узла для красно-черного дерева.

4.10.1 Подробное описание

Узел дерева поиска.

Узел хранит строковый ключ и динамический массив объектов с этим ключом. Такой подход позволяет корректно обрабатывать неunikальные ключи.

См. определение в файле [search_algorithms.h](#) строка 161

4.10.2 Конструктор(ы)

4.10.2.1 TreeNode()

```
TreeNode::TreeNode (  
    DataObject * obj)
```

Конструктор узла дерева.

Аргументы

obj	Объект, по которому создается узел.
-----	-------------------------------------

См. определение в файле [search_algorithms.cpp](#) строка 88

4.10.2.2 ~TreeNode()

```
TreeNode::~~TreeNode ()
```

Деструктор узла дерева.

См. определение в файле [search_algorithms.cpp](#) строка 98

4.10.3 Данные класса

4.10.3.1 isRed

```
bool TreeNode::isRed
```

Цвет узла для красно-черного дерева.

См. определение в файле [search_algorithms.h](#) строка 178

4.10.3.2 key

```
string TreeNode::key
```

Ключ узла.

См. определение в файле [search_algorithms.h](#) строка 163

4.10.3.3 left

```
TreeNode\* TreeNode::left
```

Левый потомок.

См. определение в файле [search_algorithms.h](#) строка 169

4.10.3.4 objects

`DynamicArray*` `TreeNode::objects`

Массив объектов, имеющих одинаковый ключ.

См. определение в файле [search_algorithms.h](#) строка 166

4.10.3.5 parent

`TreeNode*` `TreeNode::parent`

Родительский узел.

См. определение в файле [search_algorithms.h](#) строка 175

4.10.3.6 right

`TreeNode*` `TreeNode::right`

Правый потомок.

См. определение в файле [search_algorithms.h](#) строка 172

Объявления и описания членов структур находятся в файлах:

- [search_algorithms.h](#)
- [search_algorithms.cpp](#)

Глава 5

Файлы

5.1 Файл main.cpp

Главный файл программы для запуска сравнения алгоритмов поиска.

```
#include "search_algorithms.h"
```

Функции

- `int main ()`
Точка входа в программу.

5.1.1 Подробное описание

Главный файл программы для запуска сравнения алгоритмов поиска.

Файл содержит точку входа в программу. В `main` выполняется загрузка данных из файла `ds.txt`, выбор ключа поиска, запуск тестирования алгоритмов и сохранение результатов в файл `res.txt`.

См. определение в файле [main.cpp](#)

5.1.2 Функции

5.1.2.1 `main()`

```
int main ()
```

Точка входа в программу.

Выполняет загрузку исходных данных, задает размеры тестовых массивов, запускает тестирование алгоритмов поиска и освобождает выделенную память.

Возвращает

Код завершения программы.

См. определение в файле [main.cpp](#) строка 20

5.2 main.cpp

См. документацию.

```
00001
00009
00010 #include "search_algorithms.h"
00011
00020 int main() {
00021
00022     DynamicArray* data = loadDataFromFile("ds.txt");
00023
00024     cout << "Loaded: " << data->getSize() << endl;
00025
00026     string searchKey = getKeyFromData(data);
00027
00028     int sizes[] = {100, 500, 1000, 5000, 10000, 25000, 50000, 75000, 90000, 100000};
00029
00030     int numSizes = sizeof(sizes) / sizeof(sizes[0]);
00031
00032     Benchmark benchmark;
00033     benchmark.runAllTests(sizes, numSizes, searchKey, data);
00034
00035     delete data;
00036
00037     cout << "Done" << endl;
00038
00039     return 0;
00040 }
```

5.3 Файл page.md

5.4 Файл search_algorithms.cpp

Файл реализации структур данных, алгоритмов поиска и тестирования.

```
#include "search_algorithms.h"
#include <map>
#include <cstdlib>
#include <iomanip>
```

Функции

- static void `pushTreeNode` (`TreeNode **&stack`, `int &top`, `int &capacity`, `TreeNode *node`)
Добавляет узел во вспомогательный стек.
- static void `destroyTreeNodes` (`TreeNode *root`)
Удаляет дерево без использования рекурсии.
- `DynamicArray * loadDataFromFile` (`const string &filename`)
Загружает данные из файла.
- string `getKeyFromData` (`DynamicArray *data`)
Получает ключ поиска из первого объекта массива.

5.4.1 Подробное описание

Файл реализации структур данных, алгоритмов поиска и тестирования.

В файле реализованы динамический массив, бинарное дерево поиска, красно-черное дерево, хэш-таблица, загрузка данных из файла и сохранение результатов тестирования.

См. определение в файле `search_algorithms.cpp`

5.4.2 Функции

5.4.2.1 destroyTreeNodes()

```
void destroyTreeNodes (  
    TreeNode * root) [static]
```

Удаляет дерево без использования рекурсии.

Нерекурсивное удаление используется для предотвращения переполнения стека при большом количестве узлов.

Аргументы

root	Корень удаляемого дерева.
------	---------------------------

См. определение в файле [search_algorithms.cpp](#) строка 144

5.4.2.2 getKeyFromData()

```
string getKeyFromData (  
    DynamicArray * data)
```

Получает ключ поиска из первого объекта массива.

Аргументы

data	Массив данных.
------	----------------

Возвращает

Ключ поиска.

См. определение в файле [search_algorithms.cpp](#) строка 700

5.4.2.3 loadDataFromFile()

```
DynamicArray * loadDataFromFile (  
    const string & filename)
```

Загружает данные из файла.

Файл должен содержать строки формата field1;field2;value1;value2.

Аргументы

filename	Имя файла.
----------	------------

Возвращает

Массив загруженных объектов.

См. определение в файле [search_algorithms.cpp](#) строка 666

5.4.2.4 pushTreeNode()

```
void pushTreeNode (
    TreeNode **& stack,
    int & top,
    int & capacity,
    TreeNode * node) [static]
```

Добавляет узел во вспомогательный стек.

Функция используется при нерекурсивном удалении деревьев.

Аргументы

stack	Массив указателей на узлы.
top	Индекс вершины стека.
capacity	Вместимость стека.
node	Добавляемый узел.

См. определение в файле [search_algorithms.cpp](#) строка 112

5.5 search_algorithms.cpp

[См. документацию.](#)

```
00001
00009
00010 #include "search_algorithms.h"
00011 #include <map>
00012 #include <cstdlib>
00013 #include <iomanip>
00014
00015 DataObject::DataObject() : value1(0), value2(0) {
00016 }
00017
00018 DataObject::DataObject(const string& f1, const string& f2, int v1, int v2)
00019     : field1(f1), field2(f2), value1(v1), value2(v2) {
00020 }
00021
00022 string DataObject::getKey() const {
00023     return field1;
00024 }
00025
00026 DynamicArray::DynamicArray(int initialCapacity, bool ownsObjects) {
00027     capacity = initialCapacity;
00028     size = 0;
00029     this->ownsObjects = ownsObjects;
00030     arr = new DataObject * [capacity];
00031 }
00032
00033 DynamicArray::~DynamicArray() {
00034     if (ownsObjects) {
00035         for (int i = 0; i < size; i++) {
00036             delete arr[i];
00037         }
00038     }
00039
00040     delete[] arr;
00041 }
00042
00043 void DynamicArray::reserve(int newCapacity) {
00044     if (newCapacity <= capacity) {
00045         return;
00046     }
00047
00048     DataObject** newArr = new DataObject * [newCapacity];
00049
00050     for (int i = 0; i < size; i++) {
```

```

00051     newArr[i] = arr[i];
00052 }
00053
00054 delete[] arr;
00055
00056 arr = new Arr;
00057 capacity = newCapacity;
00058 }
00059
00060 void DynamicArray::push_back(DataObject* obj) {
00061     if (size >= capacity) {
00062         int newCapacity = capacity * 2;
00063
00064         DataObject** newArr = new DataObject * [newCapacity];
00065
00066         for (int i = 0; i < size; i++) {
00067             newArr[i] = arr[i];
00068         }
00069
00070         delete[] arr;
00071
00072         arr = newArr;
00073         capacity = newCapacity;
00074     }
00075
00076     arr[size] = obj;
00077     size++;
00078 }
00079
00080 DataObject* DynamicArray::operator[](int index) {
00081     return arr[index];
00082 }
00083
00084 int DynamicArray::getSize() const {
00085     return size;
00086 }
00087
00088 TreeNode::TreeNode(DataObject* obj) {
00089     key = obj->getKey();
00090     objects = new DynamicArray(2, false);
00091     objects->push_back(obj);
00092     left = nullptr;
00093     right = nullptr;
00094     parent = nullptr;
00095     isRed = true;
00096 }
00097
00098 TreeNode::~TreeNode() {
00099     delete objects;
00100 }
00101
00112 static void pushTreeNode(TreeNode**& stack, int& top, int& capacity, TreeNode* node) {
00113     if (node == nullptr) {
00114         return;
00115     }
00116
00117     if (top >= capacity) {
00118         int newCapacity = capacity * 2;
00119
00120         TreeNode** newStack = new TreeNode * [newCapacity];
00121
00122         for (int i = 0; i < top; i++) {
00123             newStack[i] = stack[i];
00124         }
00125
00126         delete[] stack;
00127
00128         stack = newStack;
00129         capacity = newCapacity;
00130     }
00131
00132     stack[top] = node;
00133     top++;
00134 }
00135
00144 static void destroyTreeNodes(TreeNode* root) {
00145     if (root == nullptr) {
00146         return;
00147     }
00148
00149     int capacity = 1024;
00150     int top = 0;
00151
00152     TreeNode** stack = new TreeNode * [capacity];
00153
00154     pushTreeNode(stack, top, capacity, root);
00155

```

```

00156     while (top > 0) {
00157         TreeNode* current = stack[top - 1];
00158         top--;
00159
00160         pushTreeNode(stack, top, capacity, current->left);
00161         pushTreeNode(stack, top, capacity, current->right);
00162
00163         delete current;
00164     }
00165
00166     delete[] stack;
00167 }
00168
00169 DynamicArray* LinearSearch::search(DynamicArray* arr, const string& key) {
00170     DynamicArray* results = new DynamicArray(10, false);
00171
00172     for (int i = 0; i < arr->getSize(); i++) {
00173         if ((*arr)[i]->getKey() == key) {
00174             results->push_back((*arr)[i]);
00175         }
00176     }
00177
00178     return results;
00179 }
00180
00181 BinarySearchTree::BinarySearchTree() {
00182     root = nullptr;
00183 }
00184
00185 BinarySearchTree::~BinarySearchTree() {
00186     destroyTreeNodes(root);
00187 }
00188
00189 void BinarySearchTree::insert(DataObject* obj) {
00190     if (root == nullptr) {
00191         root = new TreeNode(obj);
00192         return;
00193     }
00194
00195     TreeNode* current = root;
00196     TreeNode* parent = nullptr;
00197     string key = obj->getKey();
00198
00199     while (current != nullptr) {
00200         parent = current;
00201
00202         if (key == current->key) {
00203             current->objects->push_back(obj);
00204             return;
00205         }
00206
00207         if (key < current->key) {
00208             current = current->left;
00209         }
00210         else {
00211             current = current->right;
00212         }
00213     }
00214
00215     TreeNode* newNode = new TreeNode(obj);
00216     newNode->parent = parent;
00217
00218     if (key < parent->key) {
00219         parent->left = newNode;
00220     }
00221     else {
00222         parent->right = newNode;
00223     }
00224 }
00225
00226 void BinarySearchTree::buildFromArray(DynamicArray* arr) {
00227     for (int i = 0; i < arr->getSize(); i++) {
00228         insert((*arr)[i]);
00229     }
00230 }
00231
00232 DynamicArray* BinarySearchTree::search(const string& key) {
00233     DynamicArray* results = new DynamicArray(10, false);
00234
00235     TreeNode* current = root;
00236
00237     while (current != nullptr) {
00238         if (key == current->key) {
00239             for (int i = 0; i < current->objects->getSize(); i++) {
00240                 results->push_back((*current->objects)[i]);
00241             }
00242

```

```

00243         return results;
00244     }
00245
00246     if (key < current->key) {
00247         current = current->left;
00248     }
00249     else {
00250         current = current->right;
00251     }
00252 }
00253
00254 return results;
00255 }
00256
00257 RedBlackTree::RedBlackTree() {
00258     root = nullptr;
00259 }
00260
00261 RedBlackTree::~RedBlackTree() {
00262     destroyTreeNodes(root);
00263 }
00264
00265 void RedBlackTree::rotateLeft(TreeNode* x) {
00266     TreeNode* y = x->right;
00267
00268     x->right = y->left;
00269
00270     if (y->left != nullptr) {
00271         y->left->parent = x;
00272     }
00273
00274     y->parent = x->parent;
00275
00276     if (x->parent == nullptr) {
00277         root = y;
00278     }
00279     else if (x == x->parent->left) {
00280         x->parent->left = y;
00281     }
00282     else {
00283         x->parent->right = y;
00284     }
00285
00286     y->left = x;
00287     x->parent = y;
00288 }
00289
00290 void RedBlackTree::rotateRight(TreeNode* x) {
00291     TreeNode* y = x->left;
00292
00293     x->left = y->right;
00294
00295     if (y->right != nullptr) {
00296         y->right->parent = x;
00297     }
00298
00299     y->parent = x->parent;
00300
00301     if (x->parent == nullptr) {
00302         root = y;
00303     }
00304     else if (x == x->parent->right) {
00305         x->parent->right = y;
00306     }
00307     else {
00308         x->parent->left = y;
00309     }
00310
00311     y->right = x;
00312     x->parent = y;
00313 }
00314
00315 void RedBlackTree::fixInsert(TreeNode* z) {
00316     while (z != root && z->parent->isRed) {
00317         if (z->parent == z->parent->parent->left) {
00318             TreeNode* y = z->parent->parent->right;
00319
00320             if (y != nullptr && y->isRed) {
00321                 z->parent->isRed = false;
00322                 y->isRed = false;
00323                 z->parent->parent->isRed = true;
00324                 z = z->parent->parent;
00325             }
00326             else {
00327                 if (z == z->parent->right) {
00328                     z = z->parent;
00329                     rotateLeft(z);

```

```

00330         }
00331
00332         z->parent->isRed = false;
00333         z->parent->parent->isRed = true;
00334         rotateRight(z->parent->parent);
00335     }
00336 }
00337 else {
00338     TreeNode* y = z->parent->parent->left;
00339
00340     if (y != nullptr && y->isRed) {
00341         z->parent->isRed = false;
00342         y->isRed = false;
00343         z->parent->parent->isRed = true;
00344         z = z->parent->parent;
00345     }
00346     else {
00347         if (z == z->parent->left) {
00348             z = z->parent;
00349             rotateRight(z);
00350         }
00351
00352         z->parent->isRed = false;
00353         z->parent->parent->isRed = true;
00354         rotateLeft(z->parent->parent);
00355     }
00356 }
00357 }
00358
00359 root->isRed = false;
00360 }
00361
00362 void RedBlackTree::insert(DataObject* obj) {
00363     TreeNode* y = nullptr;
00364     TreeNode* x = root;
00365
00366     string key = obj->getKey();
00367
00368     while (x != nullptr) {
00369         y = x;
00370
00371         if (key == x->key) {
00372             x->objects->push_back(obj);
00373             return;
00374         }
00375
00376         if (key < x->key) {
00377             x = x->left;
00378         }
00379         else {
00380             x = x->right;
00381         }
00382     }
00383
00384     TreeNode* z = new TreeNode(obj);
00385     z->parent = y;
00386
00387     if (y == nullptr) {
00388         root = z;
00389     }
00390     else if (z->key < y->key) {
00391         y->left = z;
00392     }
00393     else {
00394         y->right = z;
00395     }
00396
00397     fixInsert(z);
00398 }
00399
00400 void RedBlackTree::buildFromArray(DynamicArray* arr) {
00401     for (int i = 0; i < arr->getSize(); i++) {
00402         insert((*arr)[i]);
00403     }
00404 }
00405
00406 DynamicArray* RedBlackTree::search(const string& key) {
00407     DynamicArray* results = new DynamicArray(10, false);
00408
00409     TreeNode* current = root;
00410
00411     while (current != nullptr) {
00412         if (key == current->key) {
00413             for (int i = 0; i < current->objects->getSize(); i++) {
00414                 results->push_back((*current->objects)[i]);
00415             }
00416

```

```

00417         return results;
00418     }
00419
00420     if (key < current->key) {
00421         current = current->left;
00422     }
00423     else {
00424         current = current->right;
00425     }
00426 }
00427
00428 return results;
00429 }
00430
00431 HashTable::HashTable(int size) {
00432     tableSize = size;
00433     table = new Bucket[tableSize];
00434     collisions = 0;
00435
00436     for (int i = 0; i < tableSize; i++) {
00437         initBucket(table[i]);
00438     }
00439 }
00440
00441 HashTable::~HashTable() {
00442     for (int i = 0; i < tableSize; i++) {
00443         delete[] table[i].data;
00444     }
00445     delete[] table;
00446 }
00447
00448 void HashTable::initBucket(Bucket& bucket) {
00449     bucket.capacity = 0;
00450     bucket.size = 0;
00451     bucket.data = nullptr;
00452 }
00453
00454 void HashTable::addToBucket(Bucket& bucket, DataObject* obj) {
00455     if (bucket.capacity == 0) {
00456         bucket.capacity = 4;
00457         bucket.data = new DataObject * [bucket.capacity];
00458     }
00459
00460     if (bucket.size >= bucket.capacity) {
00461         int newCapacity = bucket.capacity * 2;
00462         DataObject** newData = new DataObject * [newCapacity];
00463
00464         for (int i = 0; i < bucket.size; i++) {
00465             newData[i] = bucket.data[i];
00466         }
00467
00468         delete[] bucket.data;
00469
00470         bucket.data = newData;
00471         bucket.capacity = newCapacity;
00472     }
00473
00474     bucket.data[bucket.size] = obj;
00475     bucket.size++;
00476 }
00477
00478 int HashTable::hashFunction(const string& key) const {
00479     unsigned long hash = 5381;
00480
00481     for (size_t i = 0; i < key.length(); i++) {
00482         hash = ((hash << 5) + hash) + key[i];
00483     }
00484
00485     return hash % tableSize;
00486 }
00487
00488 void HashTable::insert(DataObject* obj) {
00489     int index = hashFunction(obj->getKey());
00490
00491     if (table[index].size > 0) {
00492         collisions++;
00493     }
00494
00495     addToBucket(table[index], obj);
00496 }
00497
00498 void HashTable::buildFromArray(DynamicArray* arr) {
00499     for (int i = 0; i < arr->getSize(); i++) {
00500         insert((*arr)[i]);
00501     }
00502 }

```

```

00504 }
00505
00506 DynamicArray* HashTable::search(const string& key) {
00507     DynamicArray* results = new DynamicArray(10, false);
00508
00509     int index = hashFunction(key);
00510
00511     for (int i = 0; i < table[index].size; i++) {
00512         if (table[index].data[i]->getKey() == key) {
00513             results->push_back(table[index].data[i]);
00514         }
00515     }
00516
00517     return results;
00518 }
00519
00520 int HashTable::getCollisions() const {
00521     return collisions;
00522 }
00523
00524 int Benchmark::getRepeatCount(int size) {
00525     if (size <= 1000) {
00526         return 1000;
00527     }
00528
00529     if (size <= 10000) {
00530         return 300;
00531     }
00532
00533     if (size <= 50000) {
00534         return 100;
00535     }
00536
00537     return 50;
00538 }
00539
00540 Benchmark::Result* Benchmark::runTest(DynamicArray* data, const string& searchKey) {
00541     Result* result = new Result();
00542
00543     result->size = data->getSize();
00544
00545     int repeats = getRepeatCount(data->getSize());
00546
00547     clock_t start;
00548
00549     start = clock();
00550
00551     for (int i = 0; i < repeats; i++) {
00552         DynamicArray* linearResults = LinearSearch::search(data, searchKey);
00553         delete linearResults;
00554     }
00555
00556     result->linearTime = double(clock() - start) / CLOCKS_PER_SEC * 1000 / repeats;
00557
00558     BinarySearchTree bst;
00559     bst.buildFromArray(data);
00560
00561     start = clock();
00562
00563     for (int i = 0; i < repeats; i++) {
00564         DynamicArray* bstResults = bst.search(searchKey);
00565         delete bstResults;
00566     }
00567
00568     result->bstTime = double(clock() - start) / CLOCKS_PER_SEC * 1000 / repeats;
00569
00570     RedBlackTree rbt;
00571     rbt.buildFromArray(data);
00572
00573     start = clock();
00574
00575     for (int i = 0; i < repeats; i++) {
00576         DynamicArray* rbtResults = rbt.search(searchKey);
00577         delete rbtResults;
00578     }
00579
00580     result->rbTreeTime = double(clock() - start) / CLOCKS_PER_SEC * 1000 / repeats;
00581
00582     HashTable ht(data->getSize() * 2);
00583     ht.buildFromArray(data);
00584
00585     start = clock();
00586
00587     for (int i = 0; i < repeats; i++) {
00588         DynamicArray* hashResults = ht.search(searchKey);
00589         delete hashResults;
00590     }
00591 }

```



```

00591
00592     result->hashTime = double(clock() - start) / CLOCKS_PER_SEC * 1000 / repeats;
00593     result->collisions = ht.getCollisions();
00594
00595     multimap<string, DataObject*> mmap = new multimap<string, DataObject*>();
00596
00597     for (int i = 0; i < data->getSize(); i++) {
00598         mmap->insert(pair<string, DataObject*>((*data)[i]->getKey(), (*data)[i]));
00599     }
00600
00601     volatile int controlCounter = 0;
00602
00603     start = clock();
00604
00605     for (int i = 0; i < repeats; i++) {
00606         pair<multimap<string, DataObject*>::iterator, multimap<string, DataObject*>::iterator> range =
mmap->equal_range(searchKey);
00607
00608         for (multimap<string, DataObject*>::iterator it = range.first; it != range.second; ++it) {
00609             controlCounter++;
00610         }
00611     }
00612
00613     result->multimapTime = double(clock() - start) / CLOCKS_PER_SEC * 1000 / repeats;
00614
00615     delete mmap;
00616
00617     return result;
00618 }
00619
00620 void Benchmark::runAllTests(int sizes[], int numSizes, const string& searchKey, DynamicArray* originalData) {
00621     Result** results = new Result * [numSizes];
00622
00623     for (int i = 0; i < numSizes; i++) {
00624         cout << "Rows: " << sizes[i] << endl;
00625
00626         DynamicArray* testData = new DynamicArray(sizes[i], false);
00627
00628         for (int j = 0; j < sizes[i]; j++) {
00629             testData->push_back((*originalData)[j]);
00630         }
00631
00632         results[i] = runTest(testData, searchKey);
00633
00634         delete testData;
00635     }
00636
00637     exportToTXT(results, numSizes, "res.txt");
00638
00639     for (int i = 0; i < numSizes; i++) {
00640         delete results[i];
00641     }
00642
00643     delete[] results;
00644 }
00645
00646 void Benchmark::exportToTXT(Result** results, int count, const string& filename) {
00647     ofstream file(filename.c_str());
00648
00649     file << fixed << setprecision(6);
00650
00651     file << "Раз-
мер;Линейный_поиск_мс;Бинарное_дерево_мс;Красно_черное_дерево_мс;Хэш_таблица_мс;Ассоциативный_массив_мс;Коллизии"
<< endl;
00652
00653     for (int i = 0; i < count; i++) {
00654         file << results[i]->size << ";";
00655         << results[i]->linearTime << ";";
00656         << results[i]->bstTime << ";";
00657         << results[i]->rbTreeTime << ";";
00658         << results[i]->hashTime << ";";
00659         << results[i]->multimapTime << ";";
00660         << results[i]->collisions << endl;
00661     }
00662
00663     file.close();
00664 }
00665
00666 DynamicArray* loadDataFromFile(const string& filename) {
00667     DynamicArray* data = new DynamicArray(10, true);
00668
00669     fstream file(filename.c_str(), ios::in);
00670
00671     string line;
00672
00673     while (getline(file, line)) {
00674         if (line.length() > 0 && line[line.length() - 1] == '\r') {

```

```

00675         line.erase(line.length() - 1);
00676     }
00677
00678     size_t pos1 = line.find(';');
00679     size_t pos2 = line.find(';', pos1 + 1);
00680     size_t pos3 = line.find(';', pos2 + 1);
00681
00682     string field1 = line.substr(0, pos1);
00683     string field2 = line.substr(pos1 + 1, pos2 - pos1 - 1);
00684     string val1_str = line.substr(pos2 + 1, pos3 - pos2 - 1);
00685     string val2_str = line.substr(pos3 + 1);
00686
00687     int val1 = atoi(val1_str.c_str());
00688     int val2 = atoi(val2_str.c_str());
00689
00690     DataObject* obj = new DataObject(field1, field2, val1, val2);
00691
00692     data->push_back(obj);
00693 }
00694
00695 file.close();
00696
00697 return data;
00698 }
00699
00700 string getKeyFromData(DynamicArray* data) {
00701     return (*data)[0]->getKey();
00702 }

```

5.6 Файл search_algorithms.h

Заголовочный файл с объявлениями структур, классов и функций поиска.

```

#include <iostream>
#include <fstream>
#include <string>
#include <ctime>

```

Классы

- struct [DataObject](#)
Структура объекта данных.
- class [DynamicArray](#)
Динамический массив указателей на объекты [DataObject](#).
- struct [TreeNode](#)
Узел дерева поиска.
- class [LinearSearch](#)
Класс линейного поиска.
- class [BinarySearchTree](#)
Бинарное дерево поиска.
- class [RedBlackTree](#)
Красно-черное дерево.
- class [HashTable](#)
Хэш-таблица с методом цепочек.
- struct [HashTable::Bucket](#)
Ячейка хэш-таблицы.
- class [Benchmark](#)
Класс тестирования алгоритмов поиска.
- struct [Benchmark::Result](#)
Структура результата тестирования.

Функции

- `DynamicArray * loadDataFromFile (const string &filename)`
Загружает данные из файла.
- `string getKeyFromData (DynamicArray *data)`
Получает ключ поиска из первого объекта массива.

5.6.1 Подробное описание

Заголовочный файл с объявлениями структур, классов и функций поиска.

В файле описаны структуры данных и классы, используемые для реализации линейного поиска, бинарного дерева поиска, красно-черного дерева, хэш-таблицы и тестирования времени поиска.

См. определение в файле [search_algorithms.h](#)

5.6.2 Функции

5.6.2.1 `getKeyFromData()`

```
string getKeyFromData (  
    DynamicArray * data)
```

Получает ключ поиска из первого объекта массива.

Аргументы

<code>data</code>	Массив данных.
-------------------	----------------

Возвращает

Ключ поиска.

См. определение в файле [search_algorithms.cpp](#) строка 700

5.6.2.2 `loadDataFromFile()`

```
DynamicArray * loadDataFromFile (  
    const string & filename)
```

Загружает данные из файла.

Файл должен содержать строки формата `field1;field2;value1;value2`.

Аргументы

<code>filename</code>	Имя файла.
-----------------------	------------

Возвращает

Массив загруженных объектов.

См. определение в файле [search_algorithms.cpp](#) строка 666

5.7 search_algorithms.h

См. документацию.

```

00001
00009
00034
00035 #ifndef SEARCH_ALGORITHMS_H
00036 #define SEARCH_ALGORITHMS_H
00037
00038 #include <iostream>
00039 #include <fstream>
00040 #include <string>
00041 #include <ctime>
00042
00043 using namespace std;
00044
00051 struct DataObject {
00053     string field1;
00054
00056     string field2;
00057
00059     int value1;
00060
00062     int value2;
00063
00067     DataObject();
00068
00077     DataObject(const string& f1, const string& f2, int v1, int v2);
00078
00084     string getKey() const;
00085 };
00086
00094 class DynamicArray {
00095 private:
00097     DataObject** arr;
00098
00100     int capacity;
00101
00103     int size;
00104
00106     bool ownsObjects;
00107
00108 public:
00115     DynamicArray(int initialCapacity = 10, bool ownsObjects = false);
00116
00123     ~DynamicArray();
00124
00130     void push_back(DataObject* obj);
00131
00137     void reserve(int newCapacity);
00138
00145     DataObject* operator[](int index);
00146
00152     int getSize() const;
00153 };
00154
00161 struct TreeNode {
00163     string key;
00164
00166     DynamicArray* objects;
00167
00169     TreeNode* left;
00170
00172     TreeNode* right;
00173
00175     TreeNode* parent;
00176
00178     bool isRed;
00179
00185     TreeNode(DataObject* obj);
00186
00190     ~TreeNode();
00191 };
00192
00199 class LinearSearch {
00200 public:
00208     static DynamicArray* search(DynamicArray* arr, const string& key);
00209 };
00210
00218 class BinarySearchTree {
00219 private:
00221     TreeNode* root;
00222
00223 public:
00227     BinarySearchTree();

```

```

00228
00232     ~BinarySearchTree();
00233
00239     void insert(DataObject* obj);
00240
00246     void buildFromArray(DynamicArray* arr);
00247
00254     DynamicArray* search(const string& key);
00255 };
00256
00263 class RedBlackTree {
00264 private:
00266     TreeNode* root;
00267
00273     void rotateLeft(TreeNode* x);
00274
00280     void rotateRight(TreeNode* x);
00281
00287     void fixInsert(TreeNode* z);
00288
00289 public:
00293     RedBlackTree();
00294
00298     ~RedBlackTree();
00299
00305     void insert(DataObject* obj);
00306
00312     void buildFromArray(DynamicArray* arr);
00313
00320     DynamicArray* search(const string& key);
00321 };
00322
00330 class HashTable {
00331 private:
00338     struct Bucket {
00340         DataObject** data;
00341
00343         int size;
00344
00346         int capacity;
00347     };
00348
00350     Bucket* table;
00351
00353     int tableSize;
00354
00356     int collisions;
00357
00364     int hashFunction(const string& key) const;
00365
00371     void initBucket(Bucket& bucket);
00372
00379     void addToBucket(Bucket& bucket, DataObject* obj);
00380
00381 public:
00387     HashTable(int size);
00388
00392     ~HashTable();
00393
00399     void insert(DataObject* obj);
00400
00406     void buildFromArray(DynamicArray* arr);
00407
00414     DynamicArray* search(const string& key);
00415
00421     int getCollisions() const;
00422 };
00423
00430 class Benchmark {
00431 private:
00438     int getRepeatCount(int size);
00439
00440 public:
00446     struct Result {
00448         int size;
00449
00451         double linearTime;
00452
00454         double bstTime;
00455
00457         double rbTreeTime;
00458
00460         double hashTime;
00461
00463         double multimapTime;
00464
00466         int collisions;

```

```
00467     };
00468
00476     Result* runTest(DynamicArray* data, const string& searchKey);
00477
00486     void runAllTests(int sizes[], int numSizes, const string& searchKey, DynamicArray* originalData);
00487
00495     void exportToTXT(Result** results, int count, const string& filename);
00496 };
00497
00506 DynamicArray* loadDataFromFile(const string& filename);
00507
00514 string getKeyFromData(DynamicArray* data);
00515
00516 #endif
```

Предметный указатель

- ~BinarySearchTree
 - BinarySearchTree, [10](#)
- ~DynamicArray
 - DynamicArray, [16](#)
- ~HashTable
 - HashTable, [19](#)
- ~RedBlackTree
 - RedBlackTree, [24](#)
- ~TreeNode
 - TreeNode, [30](#)
- addToBucket
 - HashTable, [20](#)
- arr
 - DynamicArray, [17](#)
- Benchmark, [7](#)
 - exportToTXT, [8](#)
 - getRepeatCount, [8](#)
 - runAllTests, [8](#)
 - runTest, [9](#)
- Benchmark::Result, [27](#)
 - bstTime, [28](#)
 - collisions, [28](#)
 - hashTime, [28](#)
 - linearTime, [28](#)
 - multimapTime, [28](#)
 - rbTreeTime, [28](#)
 - size, [28](#)
- BinarySearchTree, [9](#)
 - ~BinarySearchTree, [10](#)
 - BinarySearchTree, [10](#)
 - buildFromArray, [10](#)
 - insert, [10](#)
 - root, [11](#)
 - search, [11](#)
- bstTime
 - Benchmark::Result, [28](#)
- buildFromArray
 - BinarySearchTree, [10](#)
 - HashTable, [20](#)
 - RedBlackTree, [25](#)
- capacity
 - DynamicArray, [17](#)
 - HashTable::Bucket, [12](#)
- collisions
 - Benchmark::Result, [28](#)
 - HashTable, [22](#)
- data
 - HashTable::Bucket, [12](#)
- DataObject, [13](#)
 - DataObject, [13](#)
 - field1, [14](#)
 - field2, [14](#)
 - getKey, [14](#)
 - value1, [14](#)
 - value2, [15](#)
- destroyTreeNodes
 - search_algorithms.cpp, [35](#)
- DynamicArray, [15](#)
 - ~DynamicArray, [16](#)
 - arr, [17](#)
 - capacity, [17](#)
 - DynamicArray, [16](#)
 - getSize, [16](#)
 - operator[], [16](#)
 - ownsObjects, [18](#)
 - push_back, [17](#)
 - reserve, [17](#)
 - size, [18](#)
- exportToTXT
 - Benchmark, [8](#)
- field1
 - DataObject, [14](#)
- field2
 - DataObject, [14](#)
- fixInsert
 - RedBlackTree, [25](#)
- getCollisions
 - HashTable, [20](#)
- getKey
 - DataObject, [14](#)
- getKeyFromData
 - search_algorithms.cpp, [35](#)
 - search_algorithms.h, [45](#)
- getRepeatCount
 - Benchmark, [8](#)
- getSize
 - DynamicArray, [16](#)
- hashFunction
 - HashTable, [20](#)
- HashTable, [18](#)
 - ~HashTable, [19](#)
 - addToBucket, [20](#)

- buildFromArray, 20
- collisions, 22
- getCollisions, 20
- hashFunction, 20
- HashTable, 19
- initBucket, 21
- insert, 21
- search, 21
- table, 22
- tableSize, 22
- HashTable::Bucket, 11
 - capacity, 12
 - data, 12
 - size, 12
- hashTime
 - Benchmark::Result, 28
- initBucket
 - HashTable, 21
- insert
 - BinarySearchTree, 10
 - HashTable, 21
 - RedBlackTree, 25
- isRed
 - TreeNode, 30
- key
 - TreeNode, 30
- left
 - TreeNode, 30
- LinearSearch, 23
 - search, 23
- linearTime
 - Benchmark::Result, 28
- loadDataFromFile
 - search_algorithms.cpp, 35
 - search_algorithms.h, 45
- main
 - main.cpp, 33
- main.cpp, 33
 - main, 33
- multimapTime
 - Benchmark::Result, 28
- objects
 - TreeNode, 30
- operator[]
 - DynamicArray, 16
- ownsObjects
 - DynamicArray, 18
- page.md, 34
- parent
 - TreeNode, 31
- push_back
 - DynamicArray, 17
- pushTreeNode
 - search_algorithms.cpp, 35
- rbTreeTime
 - Benchmark::Result, 28
- RedBlackTree, 24
 - ~RedBlackTree, 24
 - buildFromArray, 25
 - fixInsert, 25
 - insert, 25
 - RedBlackTree, 24
 - root, 27
 - rotateLeft, 25
 - rotateRight, 26
 - search, 26
- reserve
 - DynamicArray, 17
- right
 - TreeNode, 31
- root
 - BinarySearchTree, 11
 - RedBlackTree, 27
- rotateLeft
 - RedBlackTree, 25
- rotateRight
 - RedBlackTree, 26
- runAllTests
 - Benchmark, 8
- runTest
 - Benchmark, 9
- search
 - BinarySearchTree, 11
 - HashTable, 21
 - LinearSearch, 23
 - RedBlackTree, 26
- search_algorithms.cpp, 34
 - destroyTreeNodes, 35
 - getKeyFromData, 35
 - loadDataFromFile, 35
 - pushTreeNode, 35
- search_algorithms.h, 44
 - getKeyFromData, 45
 - loadDataFromFile, 45
- size
 - Benchmark::Result, 28
 - DynamicArray, 18
 - HashTable::Bucket, 12
- table
 - HashTable, 22
- tableSize
 - HashTable, 22
- TreeNode, 29
 - ~TreeNode, 30
 - isRed, 30
 - key, 30
 - left, 30
 - objects, 30
 - parent, 31
 - right, 31
 - TreeNode, 30

value1

 DataObject, [14](#)

value2

 DataObject, [15](#)

Лабораторная работа 2, [1](#)